

计算流体力学作业 3

郑恒 2200011086

2025 年 4 月 13 日

1 问题介绍

求解一维对流方程：

$$\frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} = 0, \quad (1)$$

其中 u 为流体速度， t 为时间， x 为空间坐标。

初始条件为：

$$u(x, 0) = \sin(2\pi x), \quad x \in [0, 3]. \quad (2)$$

边界条件为周期性边界条件：

$$u(0, t) = u(3, t), \quad u(x, 0) = u(x + 3, 0). \quad (3)$$

首先可以先求解该方程的解析解。对流方程的解析解为：

$$u(x, t) = \sin(2\pi(x - t)). \quad (4)$$

下求解该方程的数值解。数值格式为显式格式，采用一阶迎风格式、Lax-Wendroff 格式和 Warming-Beam 格式进行求解。

2 数值格式介绍

2.1 一阶迎风格式 (Upwind)

一阶迎风格式是一种显式格式，其离散形式为：

$$u_j^{n+1} = u_j^n - \sigma (u_j^n - u_{j-1}^n),$$

其中 $\sigma = \frac{\Delta t}{\Delta x}$ 为 CFL 数。

为了分析该格式的精度，考虑 Taylor 展开式：

- 时间展开：

$$u_j^{n+1} = u_j^n + \Delta t u_t + \frac{\Delta t^2}{2} u_{tt} + O(\Delta t^3),$$

- 空间展开：

$$u_{j-1}^n = u_j^n - \Delta x u_x + \frac{\Delta x^2}{2} u_{xx} + O(\Delta x^3).$$

将上述展开式代入离散格式，得到：

$$\begin{aligned} u_j^n + \Delta t u_t + \frac{\Delta t^2}{2} u_{tt} + O(\Delta t^3) &= u_j^n - \sigma \left(u_j^n - \left(u_j^n - \Delta x u_x + \frac{\Delta x^2}{2} u_{xx} + O(\Delta x^3) \right) \right), \\ \Delta t u_t + \frac{\Delta t^2}{2} u_{tt} + O(\Delta t^3) &= -\sigma \left(-\Delta x u_x + \frac{\Delta x^2}{2} u_{xx} + O(\Delta x^3) \right), \\ u_t + u_x &= \frac{\Delta x}{2} (1 - \sigma) u_{xx} + O(\Delta t, \Delta x^2). \end{aligned}$$

由此可见，该格式的时间精度为 $O(\Delta t)$ ，空间精度为 $O(\Delta x)$ ，即为一阶格式。

再考虑数值格式的稳定性。设 $u_i^n = e^{i(kx - \omega t)}$ ，代入格式得到：

$$\begin{aligned} u_i^{n+1} &= u_i^n - \sigma (u_i^n - u_{i-1}^n) \\ &= e^{i(kx - \omega t)} - \sigma (e^{i(kx - \omega t)} - e^{i(k(x - \Delta x) - \omega t)}) \\ &= e^{i(kx - \omega t)} (1 - \sigma(1 - e^{-ik\Delta x})). \end{aligned}$$

稳定性的条件为：

$$|1 - \sigma(1 - e^{-ik\Delta x})| \leq 1.$$

故可得稳定性条件为：

$$|\sigma| \leq 1.$$

2.2 Lax-Wendroff 格式

Lax-Wendroff 格式是一种二阶显式格式，其离散形式为：

$$u_j^{n+1} = u_j^n - \frac{\sigma}{2} (u_{j+1}^n - u_{j-1}^n) + \frac{\sigma^2}{2} (u_{j+1}^n - 2u_j^n + u_{j-1}^n).$$

泰勒展开到 4 阶：

- 时间展开:

$$u_j^{n+1} = u_j^n + \Delta t u_t + \frac{\Delta t^2}{2} u_{tt} + \frac{\Delta t^3}{6} u_{ttt} + O(\Delta t^4),$$

- 空间展开:

$$u_{j+1}^n = u_j^n + \Delta x u_x + \frac{\Delta x^2}{2} u_{xx} + \frac{\Delta x^3}{6} u_{xxx} + O(\Delta x^4),$$

$$u_{j-1}^n = u_j^n - \Delta x u_x + \frac{\Delta x^2}{2} u_{xx} - \frac{\Delta x^3}{6} u_{xxx} + O(\Delta x^4).$$

$$\begin{aligned} u_j^n + \Delta t u_t + \frac{\Delta t^2}{2} u_{tt} + \frac{\Delta t^3}{6} u_{ttt} + O(\Delta t^4) &= u_j^n - \frac{\sigma}{2} \left(2\Delta x u_x + \frac{\Delta x^3}{3} u_{xxx} + O(\Delta x^5) \right) \\ &\quad + \frac{\sigma^2}{2} \left(\Delta x^2 u_{xx} + \frac{\Delta x^4}{12} u_{xxxx} + O(\Delta x^5) \right). \end{aligned}$$

整理后得到修正方程:

$$u_t + u_x - \frac{\Delta x^2}{6} (1 - \sigma^2) u_{xxx} + O(\Delta t^2, \Delta x^4) = 0.$$

故 lax-wendroff 格式的时间精度为 $O(\Delta t^2)$, 空间精度为 $O(\Delta x^2)$, 即精度为二阶。

再考虑数值格式的稳定性。设 $u_i^n = e^{i(kx - \omega t)}$, 代入格式得到:

$$\begin{aligned} u_i^{n+1} &= u_i^n - \frac{\sigma}{2} (u_{i+1}^n - u_{i-1}^n) + \frac{\sigma^2}{2} (u_{i+1}^n - 2u_i^n + u_{i-1}^n) \\ &= e^{i(kx - \omega t)} - \frac{\sigma}{2} (e^{i(k(x+\Delta x) - \omega t)} - e^{i(k(x-\Delta x) - \omega t)}) \\ &\quad + \frac{\sigma^2}{2} (e^{i(k(x+\Delta x) - \omega t)} - 2e^{i(kx - \omega t)} + e^{i(k(x-\Delta x) - \omega t)}) \\ &= e^{i(kx - \omega t)} \left(1 - \frac{\sigma}{2} (e^{ik\Delta x} - e^{-ik\Delta x}) + \frac{\sigma^2}{2} (e^{ik\Delta x} - 2 + e^{-ik\Delta x}) \right) \\ &= e^{i(kx - \omega t)} \left(1 - \sigma \sin(k\Delta x) + \frac{\sigma^2}{2} (e^{ik\Delta x} - 2 + e^{-ik\Delta x}) \right). \end{aligned}$$

稳定性的条件为:

$$\left| 1 - \sigma \sin(k\Delta x) + \frac{\sigma^2}{2} (e^{ik\Delta x} - 2 + e^{-ik\Delta x}) \right| \leq 1.$$

故可得稳定性条件为:

$$|\sigma| \leq 1.$$

2.3 Warming-Beam 格式

Warming-Beam 格式是一种三阶迎风格式，其离散形式为：

$$u_i^{n+1} = u_i^n - \sigma (u_i^n - u_{i-1}^n) - \frac{\sigma(1-\sigma)}{2} (u_i^n - 2u_{i-1}^n + u_{i-2}^n).$$

下证明该格式的精度为三阶。考虑 Taylor 展开式，取 $\nu = 1$ ：

- 时间展开：

$$u_i^{n+1} = u_i^n + \Delta t u_t + \frac{\Delta t^2}{2} u_{tt} + \frac{\Delta t^3}{6} u_{ttt} + O(\Delta t^4),$$

- 空间展开：

$$u_{i-1}^n = u_i^n - \Delta x u_x + \frac{\Delta x^2}{2} u_{xx} - \frac{\Delta x^3}{6} u_{xxx} + O(\Delta x^4),$$

$$u_{i-2}^n = u_i^n - 2\Delta x u_x + 2\Delta x^2 u_{xx} - \frac{4\Delta x^3}{3} u_{xxx} + O(\Delta x^4).$$

将上述展开式代入 Warming-Beam 格式：

$$\begin{aligned} u_i^{n+1} &= u_i^n - \sigma (u_i^n - u_{i-1}^n) - \frac{\sigma(1-\sigma)}{2} (u_i^n - 2u_{i-1}^n + u_{i-2}^n) \\ &= u_i^n - \sigma \left(u_i^n - \left(u_i^n - \Delta x u_x + \frac{\Delta x^2}{2} u_{xx} - \frac{\Delta x^3}{6} u_{xxx} \right) \right) \\ &\quad - \frac{\sigma(1-\sigma)}{2} \left(u_i^n - 2 \left(u_i^n - \Delta x u_x + \frac{\Delta x^2}{2} u_{xx} - \frac{\Delta x^3}{6} u_{xxx} \right) \right. \\ &\quad \left. + \left(u_i^n - 2\Delta x u_x + 2\Delta x^2 u_{xx} - \frac{4\Delta x^3}{3} u_{xxx} \right) \right) \\ &= u_i^n - \sigma \left(-\Delta x u_x + \frac{\Delta x^2}{2} u_{xx} - \frac{\Delta x^3}{6} u_{xxx} \right) \\ &\quad - \frac{\sigma(1-\sigma)}{2} (\Delta x^2 u_{xx} - \Delta x^3 u_{xxx}) \\ &= u_i^n + \sigma \Delta x u_x - \frac{\sigma \Delta x^2}{2} u_{xx} + \frac{\sigma \Delta x^3}{6} u_{xxx} \\ &\quad - \frac{\sigma(1-\sigma) \Delta x^2 u_{xx}}{2} + \frac{\sigma(1-\sigma) \Delta x^3 u_{xxx}}{2}. \end{aligned}$$

整理后得到修正方程：

$$u_t + u_x + \frac{\Delta x^2}{6} (1-\sigma)(2-\sigma) u_{xxx} + O(\Delta t^2, \Delta x^4) = 0.$$

由此可见，该格式空间精度为 $O(\Delta x^2)$ 。

再考虑数值格式的稳定性：设 $u_i^n = e^{i(kx - \omega t)}$ ，代入格式得到：

$$\begin{aligned}
u_i^{n+1} &= u_i^n - \sigma(u_i^n - u_{i-1}^n) - \frac{\sigma(1-\sigma)}{2}(u_i^n - 2u_{i-1}^n + u_{i-2}^n) \\
&= e^{i(kx - \omega t)} - \sigma(e^{i(kx - \omega t)} - e^{i(k(x-\Delta x) - \omega t)}) \\
&\quad - \frac{\sigma(1-\sigma)}{2}(e^{i(kx - \omega t)} - 2e^{i(k(x-\Delta x) - \omega t)} + e^{i(k(x-2\Delta x) - \omega t)}) \\
&= e^{i(kx - \omega t)} \left(1 - \sigma(1 - e^{-ik\Delta x}) - \frac{\sigma(1-\sigma)}{2}(1 - 2e^{-ik\Delta x} + e^{-2ik\Delta x}) \right) \\
&= e^{i(kx - \omega t)} \left(1 - \sigma(1 - e^{-ik\Delta x}) - \frac{\sigma(1-\sigma)}{2}(1 - 2e^{-ik\Delta x} + e^{-2ik\Delta x}) \right).
\end{aligned}$$

稳定性的条件为：

$$|1 - \sigma(1 - e^{-ik\Delta x}) - \frac{\sigma(1-\sigma)}{2}(1 - 2e^{-ik\Delta x} + e^{-2ik\Delta x})| \leq 1.$$

故可得稳定性条件为：

$$|\sigma| \leq 2.$$

3 代码实现

以下是实现各格式的 Python 代码:

```
1 def upwind_solver(u, nt, dx, dt):
2     """一阶迎风格式"""
3     sigma = nu * dt / dx
4     for _ in range(nt):
5         u_new = u.copy()
6         u_new[1:] = u[1:] - sigma * (u[1:] - u[:-1])
7         u_new[0] = u[0] - sigma * (u[0] - u[-1]) # 周期边界
8         u = u_new
9     return u
10
11
12 def lax_wendroff_solver(u, nt, dx, dt):
13     """Lax-Wendroff格式 (二阶)"""
14     sigma = nu * dt / dx
15     for _ in range(nt):
16         u_new = u.copy()
17
18         # 内部节点
19         u_new[1:-1] = u[1:-1] - sigma * (u[2:] - u[:-2]) /
20             2 + sigma **2 * (u[2:] - 2 * u[1:-1] + u[:-2]) /
21             2
22
23         # 周期边界处理
24         u_new[0] = u[0] - sigma * (u[1] - u[-1]) / 2 +
25             sigma **2 * (u[1] - 2 * u[0] + u[-1]) / 2
26         u_new[-1] = u[-1] - sigma * (u[0] - u[-2]) / 2 +
27             sigma **2 * (u[0] - 2 * u[-1] + u[-2]) / 2
28         u = u_new
29     return u
30
```

```

26
27 def warming_beam_solver(u, nt, dx, dt):
28     """Warming-Beam格式 (三阶迎风)"""
29     sigma = nu * dt / dx
30     for _ in range(nt):
31         u_new = u.copy()
32         # 内部节点 (三阶离散)
33         u_new[2:] = u[2:] - sigma * (u[2:] - u[1:-1]) -
            0.5 * sigma * (1 - sigma) * (u[2:] - 2 * u[1:-1]
            + u[:-2])
34         # 边界处理 (一阶迎风)
35         u_new[0] = u[0] - sigma * (u[0] - u[-1]) - 0.5 *
            sigma * (1 - sigma) * (u[0] - 2 * u[-1] + u[-2])
36         u_new[1] = u[1] - sigma * (u[1] - u[0]) - 0.5 *
            sigma * (1 - sigma) * (u[1] - 2 * u[0] + u[-1])#
            周期边界
37         u = u_new
38     return u

```

Listing 1: 数值格式实现代码

稳定性验证、精度验证、相位超前与耗散分析的代码实现如下:

```

1 # 稳定性验证
2 def validate_stability():
3     schemes = [
4         {'name': 'upwind', 'stable_sigma': 0.9, '
            unstable_sigma': 1.1},
5         {'name': 'lax_wendroff', 'stable_sigma': 0.9, '
            unstable_sigma': 1.1},
6         {'name': 'warming_beam', 'stable_sigma': 1.8, '
            unstable_sigma': 2.2}
7     ]
8
9     nx = 300

```

```

10     dx = L / nx
11     x = np.linspace(0, L, nx)
12     u_initial = np.sin(2 * np.pi * x )
13
14     for scheme in schemes:
15         plt.figure(figsize=(10, 4))
16
17         # 计算稳定和不稳定解
18         for case, sigma in [('stable solution', scheme['
19                                 stable_sigma']),
20                               ('unstable solution', scheme['
21                                   unstable_sigma'])]:
22
23             dt = sigma * dx / nu
24             nt = int(T / dt)
25
26             # 选择求解器
27             if scheme['name'] == 'upwind':
28                 u_num = upwind_solver(u_initial.copy(), nt,
29                                       dx, dt)
30             elif scheme['name'] == 'lax_wendroff':
31                 u_num = lax_wendroff_solver(u_initial.copy
32                                             (), nt, dx, dt)
33             elif scheme['name'] == 'warming_beam':
34                 u_num = warming_beam_solver(u_initial.copy
35                                             (), nt, dx, dt)
36
37             # 精确解计算
38             t_final = nt * dt
39             u_exact = exact_solution(x, t_final)
40
41             # 绘制结果
42             plt.plot(x, u_num,
43                     linestyle='-' if case == 'stable

```



```

38         solution' else ': ',
39         linewidth=2,
40         label=f'{case} (={sigma})')
41     plt.plot(x, u_exact, 'k--', alpha=0.5, label='
42         exact solution')
43
44     plt.ylim(-2, 2) # 扩大坐标范围显示不稳定震荡
45     plt.title(f"{scheme['name'].capitalize()}
46         comparison of format stability ")
47     plt.legend()
48     plt.grid(True, alpha=0.3)
49     plt.tight_layout()
50     # 添加PDF保存
51     plt.savefig(f"{scheme['name']}_stability.pdf",
52                 bbox_inches='tight', dpi=300)
53     plt.close()
54
55     plt.show()
56 # 精度验证
57 def validate_convergence():
58     plt.figure(figsize=(10, 6))
59     nx_list = [150, 300, 600, 1200, 2400] # 细化网格
60     markers = ['o', 's', 'D', '^', 'v']
61     colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '
62         #9467bd']
63     schemes = ['upwind', 'lax_wendroff', 'warming_beam']
64
65     # 定义各格式的CFL数
66     sigma_dict = {
67         'upwind': 0.8, # < 1
68         'lax_wendroff': 0.8, # < 1
69         'warming_beam': 1.8 # < 2
70     }

```

```

66     for idx, scheme in enumerate(schemes):
67         errors = []
68         dx_list = []
69         sigma = sigma_dict[scheme]
70
71     for nx in nx_list:
72         dx = L / nx
73         dt = sigma * dx / nu # 保持CFL数恒定
74         nt = int(T / dt)
75         x = np.linspace(0, L, nx, endpoint=False)
76
77         u_initial = np.sin(2 * np.pi * x)
78
79         # 计算数值解
80         if scheme == 'upwind':
81             u_num = upwind_solver(u_initial.copy(), nt,
82                                   dx, dt)
83         elif scheme == 'lax_wendroff':
84             u_num = lax_wendroff_solver(u_initial.copy(),
85                                         nt, dx, dt)
86         elif scheme == 'warming_beam':
87             u_num = warming_beam_solver(u_initial.copy(),
88                                         nt, dx, dt)
89
90         # 精确解 (波长为1)
91         t_final = nt * dt
92         x_exact = (x - nu * t_final) % L # 周期边界处
93         # 理
94         u_exact = np.sin(2 * np.pi * x_exact) # 波长=1
95
96         # 计算L2误差
97         error = np.sqrt(np.mean((u_num - u_exact) ** 2))
98     )

```

```

94         errors.append(error)
95         dx_list.append(dx)
96
97     # 收敛阶拟合 (使用后三个网格点)
98     log_dx = np.log10(dx_list[-3:])
99     log_err = np.log10(errors[-3:])
100     coeffs = np.polyfit(log_dx, log_err, 1)
101
102     plt.plot(log_dx, log_err,
103              marker=markers[idx], color=colors[idx],
104              linestyle='--',
105              label=f'{scheme} (slope={coeffs[0]:.2f})')
106
107     plt.xlabel(r'$\log_{10}(\Delta x)$', fontsize=12)
108     plt.ylabel(r'$\log_{10}(\mathrm{L2\ Error})$', fontsize=12)
109
110     plt.title('Convergence Analysis (Wavelength=1)')
111     plt.legend()
112     plt.grid(True, alpha=0.3)
113     plt.savefig("convergence_analysis.pdf", bbox_inches='
114                 tight', dpi=300)
115     plt.close()
116
117 # 耗散与相位分析
118
119 def analyze_dissipation_phase():
120     L = 3.0
121     nu = 1.0
122     T = 2.0
123     nx = 600
124     dx = L / nx
125     x = np.linspace(0, L, nx)

```

```

124     u_initial = np.sin(2 * np.pi * x)
125
126     # 定义各格式的范围（根据稳定性条件）
127     schemes = [
128         {'name': 'Upwind', 'solver': upwind_solver, '
            sigma_range': np.linspace(0.1, 0.95, 50)}, #
            < 1
129         {'name': 'Lax-Wendroff', 'solver':
            lax_wendroff_solver, 'sigma_range': np.linspace
            (0.1, 0.95, 50)}, # < 1
130         {'name': 'Warming-Beam', 'solver':
            warming_beam_solver, 'sigma_range': np.linspace
            (0.1, 1.9, 50)} # < 2
131     ]
132
133     # 绘制振幅衰减曲线
134     plt.figure(figsize=(8, 6))
135     for scheme in schemes:
136         amp_losses = []
137         valid_sigmas = []
138         for sigma in scheme['sigma_range']:
139             try:
140                 dt = sigma * dx / nu
141                 nt = int(T / dt)
142                 u_num = scheme['solver'](u_initial.copy(),
                    nt, dx, dt)
143                 amplitude = (u_num.max() - u_num.min()) / 2
144                 amp_loss = 1 - amplitude
145                 amp_losses.append(amp_loss)
146                 valid_sigmas.append(sigma)
147             except:
148                 continue # 跳过不稳定的
149     plt.plot(valid_sigmas, amp_losses, 'o-', label=

```

```

        scheme['name'])
150 plt.xlabel(r'CFL number $\sigma$')
151 plt.ylabel('Amplitude Damping Factor')
152 plt.title('Amplitude Damping vs. CFL Number')
153 plt.grid(True, alpha=0.3)
154 plt.legend()
155 plt.savefig("amplitude_damping.pdf", bbox_inches='tight
        ', dpi=300)
156 plt.close()
157
158 # 绘制相位误差曲线
159 plt.figure(figsize=(8, 6))
160 for scheme in schemes:
161     phase_errors = []
162     valid_sigmas = []
163     for sigma in scheme['sigma_range']:
164         try:
165             dt = sigma * dx / nu
166             nt = int(T / dt)
167             u_num = scheme['solver'](u_initial.copy(),
168                                     nt, dx, dt)
169             u_exact = exact_solution(x, nt * dt)
170
171             # 计算相位差
172             peak_num = x[np.argmax(u_num)]
173             x_exact_peak = x[np.argmax(u_exact)]
174             phase_diff = (peak_num - x_exact_peak) % 1
175             if phase_diff > 0.5:
176                 phase_diff = 1 - phase_diff
177             phase_errors.append(phase_diff)
178             valid_sigmas.append(sigma)
179         except:
180             continue

```

```

180         plt.plot(valid_sigmas, phase_errors, 'o-', label=
                scheme['name'])
181     plt.xlabel(r'CFL number $\sigma$')
182     plt.ylabel('Phase Error (Fraction of Wavelength)')
183     plt.title('Phase Error vs. CFL Number')
184     plt.grid(True, alpha=0.3)
185     plt.legend()
186     plt.savefig("phase_error.pdf", bbox_inches='tight', dpi
                =300)
187     plt.close()

```

Listing 2: 稳定性验证、精度验证、相位超前与耗散分析代码

其中稳定性是通过比较不同的 σ 值下数值解的表现来验证。

精度的计算方法是通过对数值解和解析解之间的 L^2 误差：

$$\text{Error} = \sqrt{\frac{\sum_{i=1}^N (u_{\text{numerical},i} - u_{\text{exact},i})^2}{N}},$$

其中 N 为网格点的总数。

然后对不同的网格密度 Δx 取对数，拟合 $\log(\text{Error})$ 与 $\log(\Delta x)$ 的线性关系。拟合直线的斜率即为数值格式的精度阶数：

$$\log(\text{Error}) = p \log(\Delta x) + C,$$

其中 p 为精度阶数， C 为常数。

耗散分析的原理是通过计算数值解的振幅衰减来评估数值格式的耗散特性。具体方法是先计算数值解的振幅：

$$\text{amplitude_num} = \frac{\max(u_{\text{num}}) - \min(u_{\text{num}})}{2},$$

由于初始振幅为 1，故定义振幅损失为：

$$\text{amplitude_loss} = 1 - \text{amplitude_num}.$$

相位超前或滞后的计算方法如下：通过数值解和解析解的峰值位置计算相位差，首先找到数值解的峰值位置 x_{num} 和解析解的峰值位置 x_{exact} ，然后计算相位差：

$$\text{phase_diff} = \frac{2\pi(x_{\text{num}} - x_{\text{exact}})}{L},$$

其中 $L = 1$ 为波长。根据相位差的正负判断相位超前或滞后:若 $\text{phase_diff} > 0$, 则为相位超前, 超前量为 phase_diff 的绝对值; 若 $\text{phase_diff} < 0$, 则为相位滞后, 滞后量为 phase_diff 的绝对值; 若 $\text{phase_diff} = 0$, 则为相位同步。

4 稳定性条件验证

数值格式的稳定性由 CFL 数 σ 决定。以下是各格式的稳定性条件:

- 一阶迎风格式: $|\sigma| \leq 1$ 。
- Lax-Wendroff 格式: $|\sigma| \leq 1$ 。
- Warming-Beam 格式: $|\sigma| \leq 2$ 。

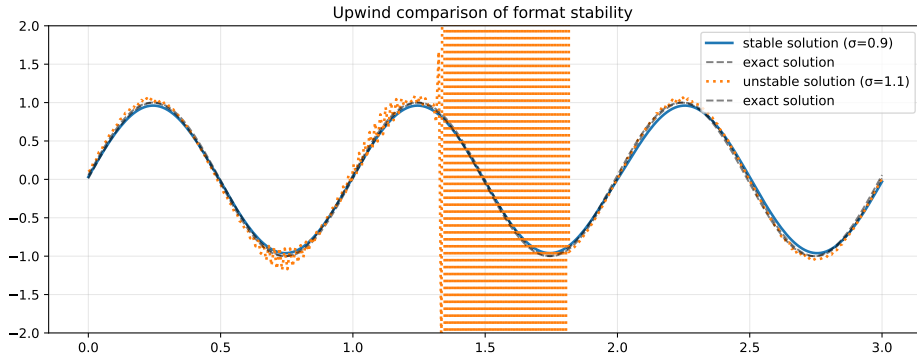


图 1: 一阶迎风格式的稳定性验证 ($\sigma \geq 1$ 时数值解不稳定)

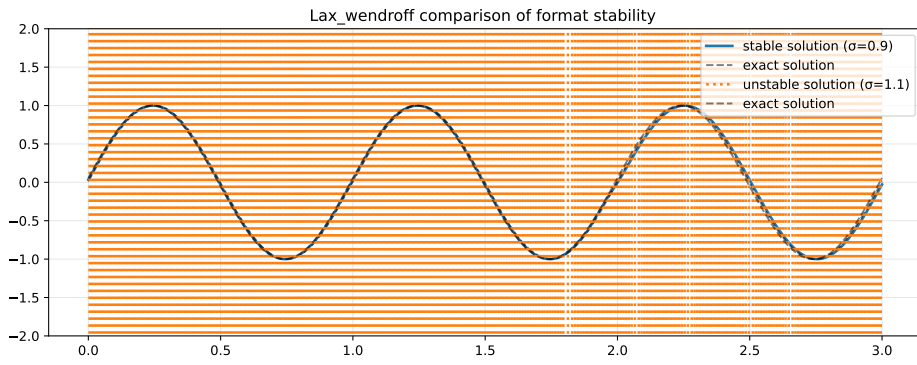


图 2: Lax-Wendroff 格式的稳定性验证 ($\sigma \geq 1$ 时数值解不稳定)

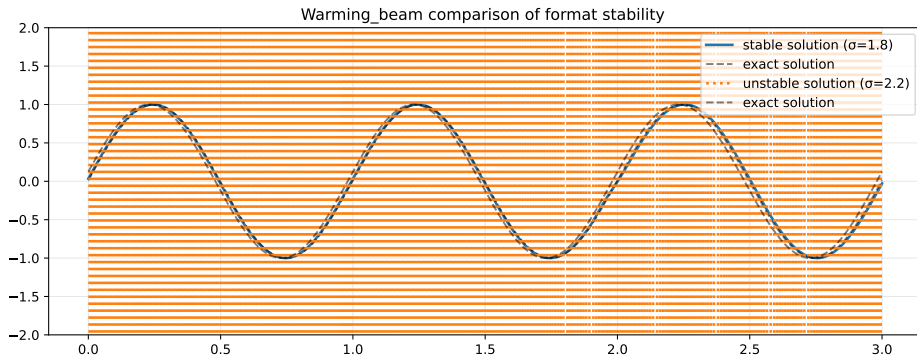


图 3: Warming-Beam 格式的稳定性验证 ($\sigma \geq 2$ 时数值解不稳定)

由图 1 可见，当 CFL 数 σ 大于 1 时，数值解（橙色）在部分 x 区域出现大幅震荡的现象，使解不稳定。

由图 2 可见，当 CFL 数 σ 大于 1 时，数值解（橙色）在全部 x 区域

出现大幅震荡的现象，使解不稳定。

由图 3 可见，当 CFL 数 σ 大于 1 时，数值解（橙色）在部分 x 区域出现大幅震荡的现象，使解不稳定。

5 精度验证

数值格式的精度通过计算数值解与解析解之间的最大绝对误差来验证。以下图 *convergence_analysis* 是各格式的精度验证结果：

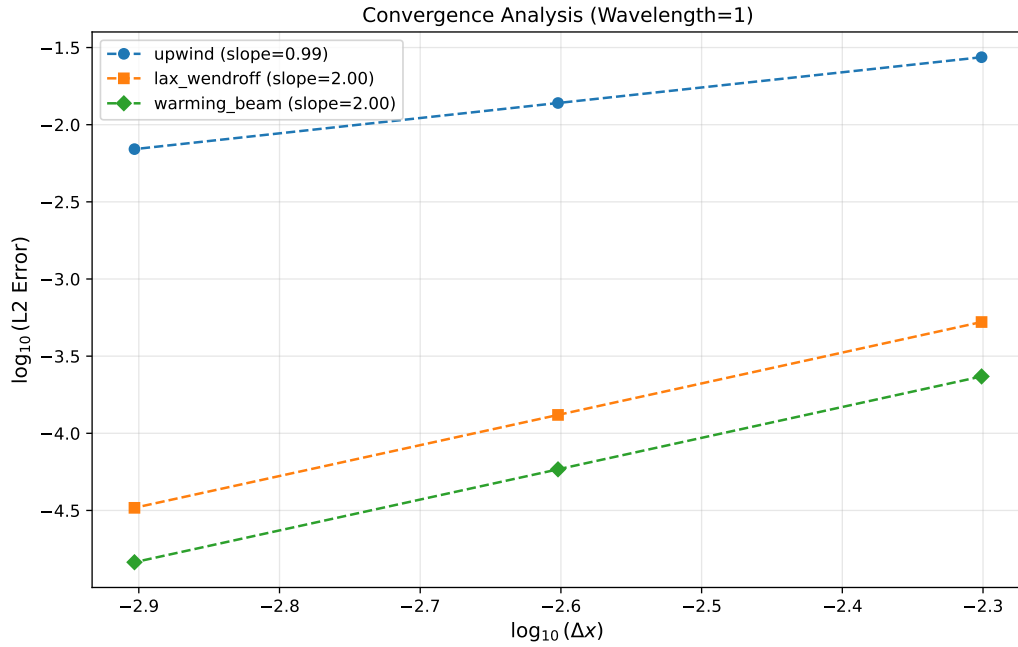


图 4: 精度验证结果

由图 4 可见，随着网格密度的增加，数值解与解析解之间的误差逐渐减小。通过对数拟合得到的斜率（即精度阶数）分别为：

- 一阶迎风格式： $p = 1$ 。
- Lax-Wendroff 格式： $p = 2$ 。
- Warming-Beam 格式： $p = 2$ 。

6 相位超前滞后与耗散分析

数值格式的相位超前滞后和耗散特性通过分析数值解与精确解的偏差来评估。以下是各格式的分析结果：

- **一阶迎风格式**：具有较强的耗散特性，振幅衰减显著，但相位偏差较小。
- **Lax-Wendroff 格式**：振幅衰减较小，相位偏差较小。
- **Warming-Beam 格式**：振幅衰减较小，相位偏差较小。

以下为各格式的耗散分析结果：

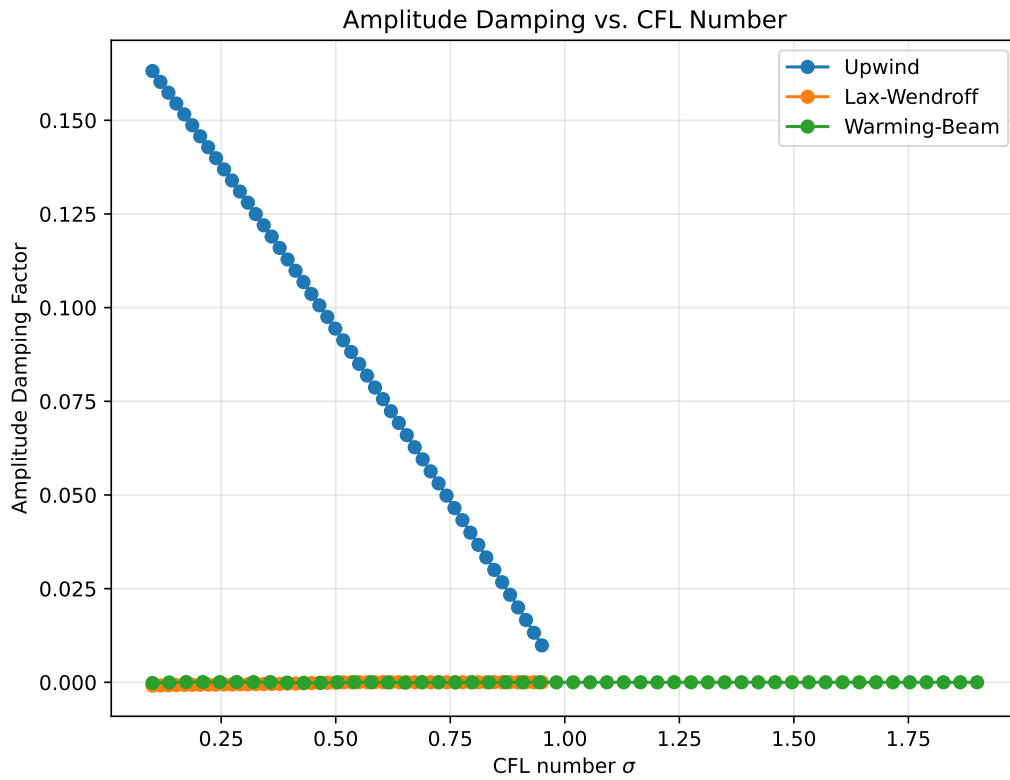


图 5: 耗散分析结果

通过图 5 可见，随着 CFL 数的减小，一阶迎风格式的数值解的振幅减小，表明一阶迎风数值格式具有耗散特性。但是 Lax-Wendroff 格式和

Warming-Beam 格式的数值解的振幅变化较小，表明这两种格式具有较好的保留特性。

再考察相位超前滞后，以下是各格式的相位超前滞后分析结果：

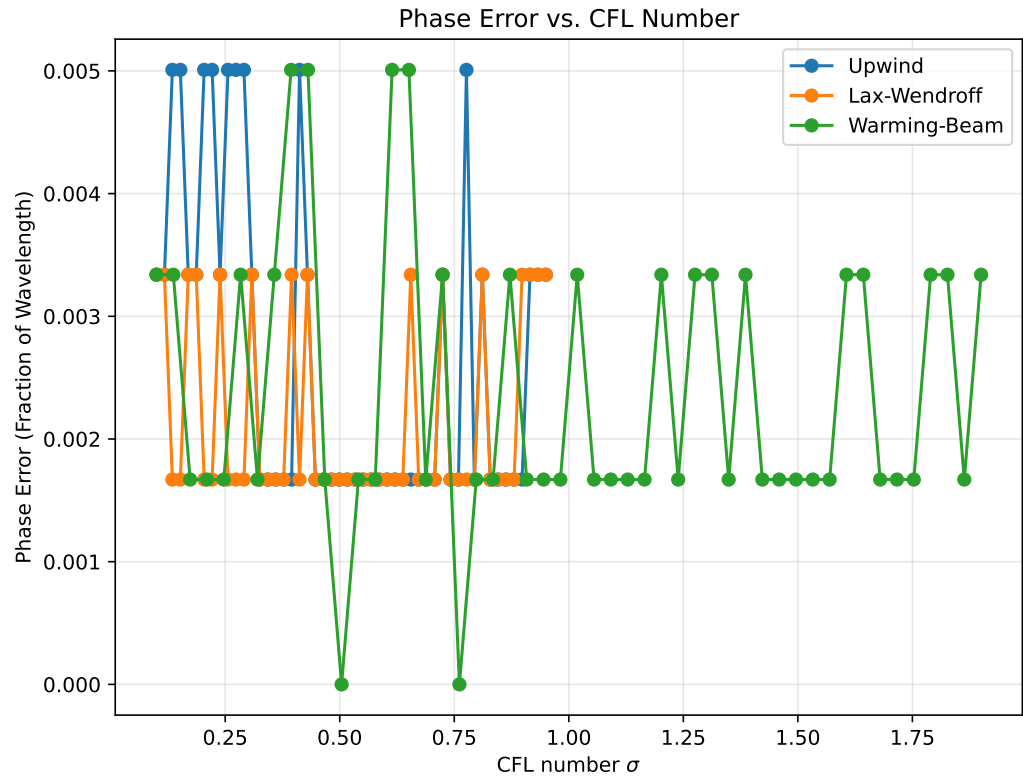


图 6: 相位超前滞后分析结果

发现三种格式的相位超前均较小，表明三种格式在相位上都具有较好的保留特性。

7 AI 工具使用说明表

AI 名称	生成代码功能	使用内容
Copilot	latex 格式框架	figure 参数调整、图片插入
Deepseek	python 绘图调整	90-104、156-166、203-210、234-241 行图绘制的具体参数调整
Deepseek	gitignore 文件忽略	全由 ai 添加

commit 记录如下图：

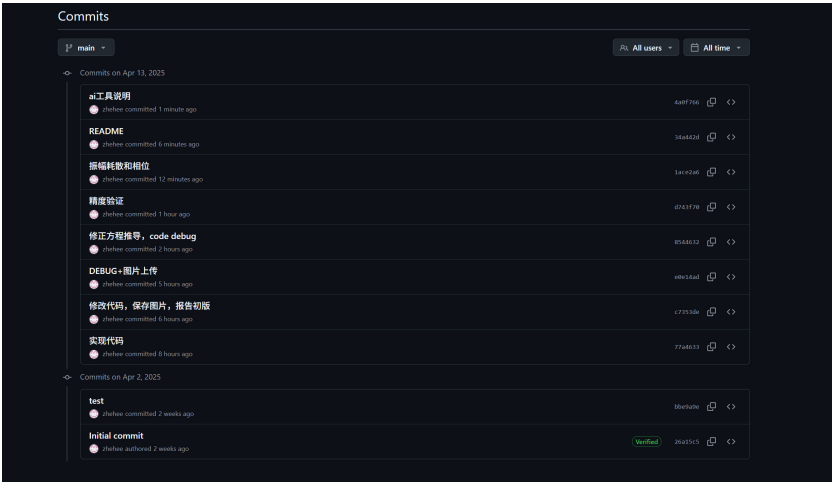


图 7: commit 记录